

LECTURE – INHERITANCE OOP(JAVA)

OBJECT ORIENTED PROGRAMMING (OOP)

MR. PREM KUMAR

LECTURER

DEPARTMENT OF CYBER SECURITY

DAWOOD UNIVERSITY OF ENGINEERING & TECHNOLOGY, KARACHI

PREM KUMAR

3/18/2026

AGENDA

- INTRODUCTION TO INHERITANCE
- TERMINOLOGY USED IN INHERITANCE
- EXTENDS KEYWORD
- TYPES OF INHERITANCE
 - EXAMPLES
- IS A RELATIONSHIP
- AGGREGATION HAS A RELATIONSHIP

INHERITANCE

- **Inheritance in Java** is a mechanism in which one object acquires all the properties and behaviors of a parent object.
- The idea behind inheritance in Java is that you can create new **classes** that are built upon existing classes.
- When you inherit from an existing class, you can reuse methods and fields of the parent class.
- Moreover, you can add new methods and fields in your current class also.

TERMINOLOGY USED IN INHERITANCE

- **Class:**
 - A class is a group of objects which have common properties.
 - It is a template or blueprint from which objects are created.
 - **Sub Class/Child Class:**
 - Subclass is a class which inherits the other class.
 - It is also called a derived class, extended class, or child class.
 - **Super Class/Parent Class:**
 - Superclass is the class from where a subclass inherits the features.
 - It is also called a base class or a parent class.
 - **Reusability:**
 - As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class.
- PREM KUMAR • You can use the same fields and methods already defined in the previous class.

EXTENDS KEYWORD

- The **extends keyword** indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.
- Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.

EXAMPLE

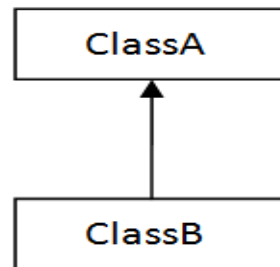
```
class Employee{  
    float salary=40000;  
}  
class Programmer extends Employee{  
    int bonus=10000;  
    public static void main(String args[]){  
        Programmer p=new Programmer();  
        System.out.println("Programmer salary is:"+p.salary);  
        System.out.println("Bonus of Programmer is:"+p.bonus);  
    }  
}
```

TYPES OF INHERITANCE

- On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.
- In java programming, multiple and hybrid inheritance is supported through interface only. We will learn about interfaces later.

SINGLE INHERITANCE

- When a class inherits another class, it is known as a *single inheritance*. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

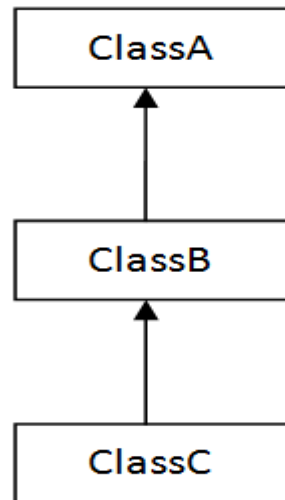


1) Single

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```


MULTILEVEL INHERITANCE EXAMPLE

- When there is a chain of inheritance, it is known as *multilevel inheritance*. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.



2) Multilevel

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}

class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

HIERARCHICAL INHERITANCE EXAMPLE

- When two or more classes inherits a single class, it is known as *hierarchical inheritance*. In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

Q) WHY MULTIPLE INHERITANCE IS NOT SUPPORTED IN JAVA?

- To reduce the complexity and simplify the language, multiple inheritance is not supported in java.
- Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.
- Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

AGGREGATION IN JAVA

- If a class have an entity reference, it is known as Aggregation. Aggregation represents HAS-A relationship.
- Consider a situation, Employee object contains many information's such as id, name, emailId etc. It contains one more object named address, which contains its own Information's such as city, state, country, zipcode etc. as given below.

WHEN USE AGGREGATION?

- Code reuse is also best achieved by aggregation when there is no is-a relationship.
- Inheritance should be used only if the relationship is-a is maintained throughout the lifetime of the objects involved; otherwise, aggregation is the best choice.

DIFFERENCE BETWEEN AGGREGATION AND ASSOCIATION

- Association and Aggregation are two important concepts in Object-Oriented Programming (OOP) and often used interchangeably. However, there is a difference between the two.
- Association refers to a relationship between two objects where one object uses the services of another object. It represents a has-a relationship, such as a teacher and a student, or a car and an engine. Association can be one-to-one, one-to-many, many-to-one, or many-to-many.
- Aggregation refers to a special type of association where an object is a part of another object. In aggregation, an object can exist independently of the other object. For example, a Department object and a Teacher object, where a teacher is a part of a department, but can also exist independently.
- In summary, Association refers to the relationship between objects and Aggregation refers to a special type of Association which represents a part-of relationship.



PREM KUMAR

THANK YOU

3/18/2026